University of Antwerp
Algorithms and Complexity
Teaching assistant: Tamara Cucumides

Exercises session 1

Turing Machines and Time Complexity

---

1 Definitions

Definition 1.1 (Turing Machine) *We define a (deterministic) Turing machine as a 7-tuple:* $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ *where*

- Q *is a finite set of states*
- Σ *is the input alphabet, where* $\vdash, \notin \Sigma$
- Γ *is the tape alphabet, where* $\Sigma \cup \{\vdash, \} \subseteq \Gamma$
- $q_0 \in Q$ *is the initial state*
- $q_{accept} \in Q$ *is the accepting state*
- $q_{reject} \in Q$ *is the rejecting state* ($q_{accept} \neq q_{reject}$)
- δ *is a partial function:*

$$\delta \quad : \quad Q \times \Gamma \to Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$$

  δ *is called the transition function*

Definition 1.2 (Time complexity) *The* **time complexity** *of a TM* M *in the worst case is defined by the function* t_M*:*

$$t_M(n) \quad = \quad \max\{\ time_M(w) \ \mid \ w \in \Sigma^* \text{ and } |w| = n\ \}.$$

where $time_M(w)$ *is the number of steps that* M *executes on input* $w \in \Sigma^*$

Definition 1.3 (Asymptotic complexity notation) *We recall the following notation:*

- **Big-O** ($\leq$)*:* $f(n) = O(g(n))$ *if:*

$$\exists c, n_0 \ s.t. \ \forall n \geq n_0 \ f(n) \leq c \cdot g(n)$$

- **Small-o** ($<$)*:* $f(n) = o(g(n))$ *if:*

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0; \ \textit{that is,}$$

$$\forall c > 0 \exists n_0 \ s.t. \ \forall \ n \geq n_0 f(n) < c \cdot g(n)$$

- **Big-Omega ($\geq$)**: $f(n) = \Omega(g(n))$ *if:*

$$g(n) = O(f(n)), \text{ that is}$$
$$\exists c, n_0 \text{ s.t. } \forall n \geq n_0 \; g(n) \leq c \cdot f(n), \text{ that is}$$
$$\exists d, n_0 \text{ s.t. } \forall n \geq n_0 \; f(n) \geq d \cdot g(n)$$

2 Session exercises

1. Let's consider a new definition of a turing machine, where in each step the head has to move either to the right or to the left (it cannot stay in the same place). Show that this can simulate any regular deterministic turing machine. What can we say about the running time?
2. Define a turing machine $\mathcal{M}$ that decides the following language

$$L = \{0^k 1^k : k \in \mathbb{N}\}$$

   (a) Implement such turing machine in https://turingmachinesimulator.com/

   (b) What is the time complexity of M?

   (c) Can we do it better than $O(n^2)$?
3. Prove if the following are true of false

   (a) $2^{n+1} = O(2^n)$

   (b) if $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then $f(n) = O(h(n))$

   (c) there exists $f(n)$ and $g(n)$ such that $f(n) = o(g(n))$ and $f(n) = \Omega(g(n))$

   (d) for every pair of functions f and g, either $f(n) = O(g(n))$ or $g(n) = O(f(n))$

3 Proposed exercises

1. Create a program in https://turingmachinesimulator.com/ that decides the following language

$$L = \{ \; w \cdot \# \cdot w^{-1} \;\; | \; w \in \{0,1\}^* \;\; \text{and} \; |w| \leq 4\}.$$

   Here w^{-1} denotes the reverse of the word w, and $|w|$ denotes the length of w. Provide a big-O analysis of your turing machine.
2. Prove if the following are true or false
   - $2^{2n} = O(2^n)$
   - if $f(n) = O(g(n))$ and $g(n) = O(f(n))$ then $f(n) = g(n)$
   - if $f(n) = O(n)$, then $2^{f(n)} = O(2^n)$